
YOHPAL LIVE CREATOR STUDIO RELEASE 1.1A-RC1 Core Camera, Timeline & Draft Pilot — Implementation Pack

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Tue, 14 Jul at 11:31

YOHPAL LIVE CREATOR STUDIO RELEASE 1.1A-RC1

Core Camera, Timeline & Draft Pilot — Implementation Pack

This RC1 activates only the verified launch foundation:

- Camera recording
- Multi-clip recording
- Gallery import
- Trim, split, delete and reorder
- Undo and redo
- Draft autosave and recovery
- Local rendering
- Upload progress, cancel and retry
- Mandatory publication gates

The following remain disabled:

- AI edit execution
- Automatic captions
- Music
- Voice-over
- Jobs, Hustle, Market and Course objects

1. RC1 Feature Configuration

apps/mobile_flutter/lib/core/config/creator_studio_flags.dart

```
final class CreatorStudioFlags {
  const CreatorStudioFlags._();

  static const bool enabled = bool.fromEnvironment(
    'CREATOR_STUDIO_ENABLED',
    defaultValue: false,
  );

  static const bool cameraEnabled = bool.fromEnvironment(
    'CREATOR_CAMERA_ENABLED',
    defaultValue: false,
  );

  static const bool galleryImportEnabled = bool.fromEnvironment(
    'CREATOR_GALLERY_IMPORT_ENABLED',
    defaultValue: false,
  );

  static const bool multiClipEnabled = bool.fromEnvironment(
    'CREATOR_MULTI_CLIP_ENABLED',
    defaultValue: false,
  );

  static const bool localRenderingEnabled = bool.fromEnvironment(
    'CREATOR_LOCAL_RENDERING_ENABLED',
    defaultValue: false,
  );

  static const bool uploadEnabled = bool.fromEnvironment(
    'CREATOR_UPLOAD_ENABLED',
    defaultValue: false,
  );

  static const bool captionsEnabled = bool.fromEnvironment(
    'CREATOR_CAPTIONS_ENABLED',
```

```

    defaultValue: false,
  );

static const bool musicEnabled = bool.fromEnvironment(
  'CREATOR_MUSIC_ENABLED',
  defaultValue: false,
);

static const bool voiceOverEnabled = bool.fromEnvironment(
  'CREATOR_VOICE_OVER_ENABLED',
  defaultValue: false,
);

static const bool ecosystemObjectsEnabled = bool.fromEnvironment(
  'CREATOR_OBJECTS_ENABLED',
  defaultValue: false,
);

static const bool aiEditExecutionEnabled = bool.fromEnvironment(
  'CREATOR_AI_EDIT_EXECUTION_ENABLED',
  defaultValue: false,
);

static const bool publicationGateRequired = bool.fromEnvironment(
  'CREATOR_PUBLICATION_GATE_REQUIRED',
  defaultValue: true,
);

static void validateRc1() {
  if (!enabled) {
    return;
  }

  if (!cameraEnabled ||
    !galleryImportEnabled ||
    !multiClipEnabled ||
    !localRenderingEnabled ||
    !uploadEnabled) {
    throw StateError(
      'Creator Studio RC1 core features must be enabled together.',
    );
  }

  if (captionsEnabled ||
    musicEnabled ||
    voiceOverEnabled ||
    ecosystemObjectsEnabled ||
    aiEditExecutionEnabled) {
    throw StateError(
      'Uncertified advanced Creator Studio features are enabled in RC1.',
    );
  }

  if (!publicationGateRequired) {
    throw StateError(
      'Creator Studio RC1 requires publication gates.',
    );
  }
}
}

```

RC1 build command

```

flutter build apk \
  --release \
  --dart-define=CREATOR_STUDIO_ENABLED=true \
  --dart-define=CREATOR_CAMERA_ENABLED=true \
  --dart-define=CREATOR_GALLERY_IMPORT_ENABLED=true \
  --dart-define=CREATOR_MULTI_CLIP_ENABLED=true \
  --dart-define=CREATOR_LOCAL_RENDERING_ENABLED=true \
  --dart-define=CREATOR_UPLOAD_ENABLED=true \
  --dart-define=CREATOR_CAPTIONS_ENABLED=false \
  --dart-define=CREATOR_MUSIC_ENABLED=false \
  --dart-define=CREATOR_VOICE_OVER_ENABLED=false \
  --dart-define=CREATOR_OBJECTS_ENABLED=false \
  --dart-define=CREATOR_AI_EDIT_EXECUTION_ENABLED=false \
  --dart-define=CREATOR_PUBLICATION_GATE_REQUIRED=true

```

Use equivalent defines for the iOS build.

2. RC1 Application Bootstrap

apps/mobile_flutter/lib/features/creator_studio/creator_studio_bootstrap.dart

```
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/widgets.dart';
import 'package:hive_ce_flutter/hive_flutter.dart';

import '../core/config/creator_studio_flags.dart';
import 'data/firebase_upload_repository.dart';
import 'data/hive_creator_project_repository.dart';
import 'services/creator_render_service.dart';
import 'services/ffmpeg_executor.dart';

final class CreatorStudioDependencies {
  const CreatorStudioDependencies({
    required this.projects,
    required this.uploads,
    required this.renderer,
  });

  final HiveCreatorProjectRepository projects;
  final FirebaseUploadRepository uploads;
  final CreatorRenderService renderer;
}

Future<CreatorStudioDependencies>
  initializeCreatorStudioDependencies() async {
  CreatorStudioFlags.validateRcl();

  await Hive.initFlutter();

  final projects = HiveCreatorProjectRepository();
  await projects.initialize();

  final uploads = FirebaseUploadRepository();

  final renderer = DefaultCreatorRenderService(
    executor: PlatformFfmpegExecutor(),
  );

  return CreatorStudioDependencies(
    projects: projects,
    uploads: uploads,
    renderer: renderer,
  );
}

String requireCreatorId() {
  final user = FirebaseAuth.instance.currentUser;

  if (user == null) {
    throw StateError(
      'An authenticated user is required to use Creator Studio.',
    );
  }

  return user.uid;
}

class CreatorStudioBootstrapError extends StatelessWidget {
  const CreatorStudioBootstrapError({
    required this.error,
    super.key,
  });

  final Object error;

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Text(
        'Creator Studio could not start.\n$error',
        textAlign: TextAlign.center,
      ),
    );
  }
}
```

3. Gallery Import

apps/mobile_flutter/lib/features/creator_studio/services/gallery_import_

service.dart

```
import 'dart:io';

import 'package:image_picker/image_picker.dart';
import 'package:uuid/uuid.dart';
import 'package:video_player/video_player.dart';

import '../domain/video_clip_edit.dart';

abstract interface class GalleryImportService {
  Future<VideoClipEdit?> selectVideo();
}

final class ImagePickerGalleryImportService
  implements GalleryImportService {
  ImagePickerGalleryImportService({
    ImagePicker? picker,
    Uuid uuid = const Uuid(),
  }) : _picker = picker ?? ImagePicker(),
       _uuid = uuid;

  final ImagePicker _picker;
  final Uuid _uuid;

  @override
  Future<VideoClipEdit?> selectVideo() async {
    final selected = await _picker.pickVideo(
      source: ImageSource.gallery,
    );

    if (selected == null) {
      return null;
    }

    final file = File(selected.path);

    if (!await file.exists()) {
      throw FileSystemException(
        'The selected video could not be found.',
        selected.path,
      );
    }

    final size = await file.length();

    if (size > 500 * 1024 * 1024) {
      throw StateError(
        'The selected video exceeds the 500 MB limit.',
      );
    }

    final controller = VideoPlayerController.file(file);

    try {
      await controller.initialize().timeout(
        const Duration(seconds: 15),
      );

      final durationMs =
        controller.value.duration.inMilliseconds;

      if (durationMs <= 0) {
        throw StateError(
          'The selected video has an invalid duration.',
        );
      }

      return VideoClipEdit(
        id: _uuid.v4(),
        sourcePath: file.path,
        originalDurationMs: durationMs,
        trimStartMs: 0,
        trimEndMs: durationMs,
      );
    } finally {
      await controller.dispose();
    }
  }
}
```

4. Editor Session Controller

This controller coordinates editing, autosave, rendering and recovery.

apps/mobile_flutter/lib/features/creator_studio/ controllers/creator_studio_session_controller.dart

```
import 'dart:async';

import 'package:flutter/foundation.dart';
import 'package:path_provider/path_provider.dart';

import '../data/creator_project_repository.dart';
import '../domain/creator_project.dart';
import '../domain/creator_project_status.dart';
import '../domain/publication_gate.dart';
import '../services/creator_render_service.dart';
import '../services/gallery_import_service.dart';
import 'creator_editor_controller.dart';

enum CreatorStudioSessionState {
  idle,
  loading,
  editing,
  rendering,
  rendered,
  failed,
}

final class CreatorStudioSessionController
  extends ChangeNotifier {
  CreatorStudioSessionController({
    required CreatorProjectRepository projects,
    required GalleryImportService gallery,
    required CreatorRenderService renderer,
    VideoPublicationGate publicationGate =
      const VideoPublicationGate(),
  }) : _projects = projects,
      _gallery = gallery,
      _renderer = renderer,
      _publicationGate = publicationGate;

  final CreatorProjectRepository _projects;
  final GalleryImportService _gallery;
  final CreatorRenderService _renderer;
  final VideoPublicationGate _publicationGate;

  CreatorProject? _project;
  CreatorEditorController? _editor;
  CreatorStudioSessionState _state =
    CreatorStudioSessionState.idle;

  double _renderProgress = 0;
  Object? _error;
  StreamSubscription<double>? _renderSubscription;
  Timer? _autosaveDebounce;

  CreatorProject? get project => _project;
  CreatorEditorController? get editor => _editor;
  CreatorStudioSessionState get state => _state;
  double get renderProgress => _renderProgress;
  Object? get error => _error;

  Future<void> load(CreatorProject project) async {
    _state = CreatorStudioSessionState.loading;
    notifyListeners();

    _project = project;
    _attachEditor(project);

    _state = CreatorStudioSessionState.editing;
    notifyListeners();
  }

  void _attachEditor(CreatorProject project) {
    _editor?.removeListener(_handleEditorChange);

    _editor = CreatorEditorController(
      initialProject: project,
    );

    _editor!.addListener(_handleEditorChange);
  }

  void _handleEditorChange() {
    final next = _editor?.project;
```

```

    if (next == null) {
        return;
    }

    _project = next.copyWith(
        status: CreatorProjectStatus.editing,
    );

    _autosaveDebounce?.cancel();

    _autosaveDebounce = Timer(
        const Duration(milliseconds: 600),
        _saveDraft,
    );

    notifyListeners();
}

Future<void> importFromGallery() async {
    try {
        final clip = await _gallery.selectVideo();

        if (clip == null) {
            return;
        }

        final current = _requireProject();
        final clips = [...current.clips, clip];

        _editor?.replaceClips(clips);
    } catch (error) {
        _error = error;
        _state = CreatorStudioSessionState.failed;
        notifyListeners();
        rethrow;
    }
}

Future<void> addRecordedClips(
    List<VideoClipEdit> recordedClips,
) async {
    if (recordedClips.isEmpty) {
        return;
    }

    final current = _requireProject();

    _editor?.replaceClips([
        ...current.clips,
        ...recordedClips,
    ]);
}

Future<void> _saveDraft() async {
    final current = _project;

    if (current == null) {
        return;
    }

    await _projects.save(
        current.copyWith(
            status: CreatorProjectStatus.draft,
            updatedAt: DateTime.now(),
        ),
    );
}

Future<void> saveImmediately() async {
    _autosaveDebounce?.cancel();
    await _saveDraft();
}

Future<void> render() async {
    final current = _requireProject();

    if (current.clips.isEmpty) {
        throw StateError(
            'At least one clip is required before rendering.',
        );
    }

    _state = CreatorStudioSessionState.rendering;
    _renderProgress = 0;
    _error = null;
    notifyListeners();
}

```

```

await _renderSubscription?.cancel();

_renderSubscription = _renderer.progress.listen(
  (progress) {
    _renderProgress = progress.clamp(0, 1);
    notifyListeners();
  },
);

try {
  final directory =
    await getApplicationDocumentsDirectory();

  final outputPath =
    '${directory.path}/creator-${current.id}.mp4';

  final renderedPath = await _renderer.render(
    project: current,
    outputPath: outputPath,
  );

  _project = current.copyWith(
    status: CreatorProjectStatus.rendered,
    renderedPath: renderedPath,
  );

  await _projects.save(_project!);

  _attachEditor(_project!);

  _state = CreatorStudioSessionState.rendered;
  _renderProgress = 1;
  notifyListeners();
} catch (error) {
  _error = error;
  _state = CreatorStudioSessionState.failed;

  _project = current.copyWith(
    status: CreatorProjectStatus.failed,
  );

  await _projects.save(_project!);
  notifyListeners();

  rethrow;
}
}

Future<void> cancelRender() async {
  await _renderer.cancel();

  _state = CreatorStudioSessionState.editing;
  _renderProgress = 0;
  notifyListeners();
}

PublicationGateResult previewPublicationGate() {
  final current = _requireProject();

  return _publicationGate.evaluate(
    PublicationGateInput(
      hasRenderedVideo:
        current.renderedPath?.isNotEmpty == true,
      uploadComplete:
        current.remoteVideoUrl?.isNotEmpty == true,
      moderationPassed: false,
      copyrightPassed: false,
      metadataComplete:
        current.title.trim().isNotEmpty &&
        current.description.trim().isNotEmpty,
      captionReviewComplete: true,
      validTimelineObjects: true,
      aiChecklistRequired: false,
      aiChecklistComplete: false,
    ),
  );
}

CreatorProject _requireProject() {
  final current = _project;

  if (current == null) {
    throw StateError(
      'Creator Studio project has not been loaded.',
    );
  }
}

```

```

    return current;
  }

  @override
  void dispose() {
    _autosaveDebounce?.cancel();
    _renderSubscription?.cancel();
    _editor?.removeListener(_handleEditorChange);
    _editor?.dispose();
    super.dispose();
  }
}

```

Required import:

```
import '../domain/video_clip_edit.dart';
```

5. Draft Recovery

apps/mobile_flutter/lib/features/creator_studio/controllers/creator_draft_controller.dart

```

import 'package:flutter/foundation.dart';

import '../data/creator_project_repository.dart';
import '../domain/creator_project.dart';

final class CreatorDraftController extends ChangeNotifier {
  CreatorDraftController({
    required CreatorProjectRepository projects,
  }) : _projects = projects;

  final CreatorProjectRepository _projects;

  List<CreatorProject> _drafts = const [];
  bool _loading = false;
  Object? _error;

  List<CreatorProject> get drafts => _drafts;
  bool get loading => _loading;
  Object? get error => _error;

  Future<void> load(String ownerId) async {
    _loading = true;
    _error = null;
    notifyListeners();

    try {
      _drafts = await _projects.findByOwner(ownerId);
    } catch (error) {
      _error = error;
    } finally {
      _loading = false;
      notifyListeners();
    }
  }

  Future<void> deleteDraft(String projectId) async {
    await _projects.delete(projectId);

    _drafts = _drafts
      ..where((draft) => draft.id != projectId)
      .toList(growable: false);

    notifyListeners();
  }
}

```

6. Creator Studio Home Screen

apps/mobile_flutter/lib/features/creator_studio/presentation/creator_studio_home_screen.dart

```

import 'package:flutter/material.dart';

import '../controllers/creator_draft_controller.dart';
import '../domain/creator_objective.dart';
import '../domain/creator_project.dart';

```

```

class CreatorStudioHomeScreen extends StatefulWidget {
  const CreatorStudioHomeScreen({
    required this.ownerId,
    required this.drafts,
    required this.onCreate,
    required this.onOpen,
    super.key,
  });

  final String ownerId;
  final CreatorDraftController drafts;

  final Future<void> Function(
    String title,
    CreatorObjective objective,
  ) onCreate;

  final void Function(CreatorProject project) onOpen;

  @override
  State<CreatorStudioHomeScreen> createState() =>
    _CreatorStudioHomeScreenState();
}

class _CreatorStudioHomeScreenState
  extends State<CreatorStudioHomeScreen> {
  @override
  void initState() {
    super.initState();
    widget.drafts.addListener(_refresh);
    widget.drafts.load(widget.ownerId);
  }

  void _refresh() {
    if (mounted) {
      setState(() {});
    }
  }

  @override
  void dispose() {
    widget.drafts.removeListener(_refresh);
    super.dispose();
  }

  Future<void> _newProject() async {
    final titleController = TextEditingController();

    final result = await showDialog<String>(
      context: context,
      builder: (context) {
        return AlertDialog(
          title: const Text('New creator project'),
          content: TextField(
            controller: titleController,
            autofocus: true,
            decoration: const InputDecoration(
              labelText: 'Project title',
            ),
          ),
          actions: [
            TextButton(
              onPressed: () =>
                Navigator.of(context).pop(),
              child: const Text('Cancel'),
            ),
            FilledButton(
              onPressed: () => Navigator.of(context).pop(
                titleController.text.trim(),
              ),
              child: const Text('Create'),
            ),
          ],
        );
      },
    );

    if (result == null || result.isEmpty) {
      return;
    }

    await widget.onCreate(
      result,
      CreatorObjective.entertain,
    );
  }
}

```

```

@override
Widget build(BuildContext context) {
  final drafts = widget.drafts.drafts;

  return Scaffold(
    appBar: AppBar(
      title: const Text('Creator Studio'),
    ),
    floatingActionButton: FloatingActionButton.extended(
      onPressed: _newProject,
      icon: const Icon(Icons.add),
      label: const Text('New project'),
    ),
    body: widget.drafts.loading
      ? const Center(
          child: CircularProgressIndicator(),
        )
      : drafts.isEmpty
        ? const Center(
            child: Text(
              'Create your first YohPal video project.',
            ),
          )
        : ListView.separated(
            padding: const EdgeInsets.all(16),
            itemCount: drafts.length,
            separatorBuilder: (_, __) =>
              const SizedBox(height: 10),
            itemBuilder: (context, index) {
              final project = drafts[index];

              return Card(
                child: ListTile(
                  leading:
                    const Icon(Icons.video_library),
                  title: Text(project.title),
                  subtitle: Text(
                    '${project.clips.length} clips • '
                    '${project.status.name}',
                  ),
                  onTap: () =>
                    widget.onOpen(project),
                  trailing: IconButton(
                    tooltip: 'Delete draft',
                    onPressed: () =>
                      widget.drafts.deleteDraft(
                        project.id,
                      ),
                  icon:
                    const Icon(Icons.delete_outline),
                ),
              ),
            ),
          );
    },
  );
}

```

7. Core Editor Screen

**apps/mobile_flutter/lib/features/creator_studio/
presentation/creator_editor_screen.dart**

```

import 'package:flutter/material.dart';

import '../controllers/creator_studio_session_controller.dart';
import '../domain/video_clip_edit.dart';
import 'widgets/publication_gate_panel.dart';
import 'widgets/timeline_strip.dart';

class CreatorEditorScreen extends StatefulWidget {
  const CreatorEditorScreen({
    required this.session,
    required this.onOpenCamera,
    required this.onContinueToUpload,
    super.key,
  });

  final CreatorStudioSessionController session;
  final VoidCallback onOpenCamera;

```

```

final VoidCallback onContinueToUpload;

@override
State<CreatorEditorScreen> createState() =>
  _CreatorEditorScreenState();
}

class _CreatorEditorScreenState
  extends State<CreatorEditorScreen> {
  String? _selectedClipId;

  @override
  void initState() {
    super.initState();
    widget.session.addListener(_refresh);
  }

  void _refresh() {
    if (mounted) {
      setState(() {});
    }
  }

  @override
  void dispose() {
    widget.session
      ..removeListener(_refresh)
      ..saveImmediately();

    super.dispose();
  }

  Future<void> _trimClip(VideoClipEdit clip) async {
    final range = await showDialog<RangeValues>(
      context: context,
      builder: (context) {
        var values = RangeValues(
          clip.trimStartMs.toDouble(),
          clip.trimEndMs.toDouble(),
        );

        return StatefulBuilder(
          builder: (context, setDialogState) {
            return AlertDialog(
              title: const Text('Trim clip'),
              content: RangeSlider(
                values: values,
                min: 0,
                max: clip.originalDurationMs.toDouble(),
                labels: RangeLabels(
                  '${(values.start / 1000).toStringAsFixed(1)}s',
                  '${(values.end / 1000).toStringAsFixed(1)}s',
                ),
                onChanged: (next) {
                  setDialogState(() {
                    values = next;
                  });
                },
              ),
              actions: [
                TextButton(
                  onPressed: () =>
                    Navigator.of(context).pop(),
                  child: const Text('Cancel'),
                ),
                FilledButton(
                  onPressed: () =>
                    Navigator.of(context).pop(values),
                  child: const Text('Apply'),
                ),
              ],
            );
          },
        );
      },
    );

    if (range == null) {
      return;
    }

    widget.session.editor?.trimClip(
      clipId: clip.id,
      startMs: range.start.round(),
      endMs: range.end.round(),
    );
  }
}

```

```

Future<void> _splitClip(VideoClipEdit clip) async {
  final splitAt = clip.trimStartMs +
    ((clip.trimEndMs - clip.trimStartMs) ~/ 2);

  widget.session.editor?.splitClip(
    clipId: clip.id,
    splitAtMs: splitAt,
  );
}

@override
Widget build(BuildContext context) {
  final project = widget.session.project;
  final editor = widget.session.editor;

  if (project == null || editor == null) {
    return const Scaffold(
      body: Center(
        child: CircularProgressIndicator(),
      ),
    );
  }

  final selected = project.clips
    .where((clip) => clip.id == _selectedClipId)
    .firstOrNull;

  return Scaffold(
    appBar: AppBar(
      title: Text(project.title),
      actions: [
        IconButton(
          tooltip: 'Undo',
          onPressed: editor.undo,
          icon: const Icon(Icons.undo),
        ),
        IconButton(
          tooltip: 'Redo',
          onPressed: editor.redo,
          icon: const Icon(Icons.redo),
        ),
        IconButton(
          tooltip: 'Save draft',
          onPressed: widget.session.saveImmediately,
          icon: const Icon(Icons.save_outlined),
        ),
      ],
    ),
    body: Column(
      children: [
        Expanded(
          child: Container(
            color: Colors.black,
            alignment: Alignment.center,
            child: const Icon(
              Icons.play_circle_outline,
              color: Colors.white,
              size: 72,
            ),
          ),
        ),
        SizedBox(
          height: 155,
          child: TimelineStrip(
            clips: project.clips,
            selectedClipId: _selectedClipId,
            onSelect: (clip) {
              setState(() {
                _selectedClipId = clip.id;
              });
            },
            onDelete: editor.deleteClip,
            onReorder: editor.reorderClip,
          ),
        ),
        if (selected != null)
          Padding(
            padding: const EdgeInsets.symmetric(
              horizontal: 12,
            ),
            child: Wrap(
              spacing: 8,
              children: [
                OutlinedButton.icon(
                  onPressed: () => _trimClip(selected),
                  icon: const Icon(Icons.content_cut),
                ),
              ],
            ),
          ),
      ],
    ),
  );
}

```

```

        label: const Text('Trim'),
      ),
      OutlinedButton.icon(
        onPressed: () => _splitClip(selected),
        icon: const Icon(Icons.call_split),
        label: const Text('Split'),
      ),
    ],
  ),
),
if (widget.session.state ==
  CreatorStudioSessionState.rendering)
  Padding(
    padding: const EdgeInsets.all(16),
    child: Column(
      children: [
        LinearProgressIndicator(
          value: widget.session.renderProgress,
        ),
        TextButton(
          onPressed: widget.session.cancelRender,
          child: const Text('Cancel render'),
        ),
      ],
    ),
  ),
),
),
Padding(
  padding: const EdgeInsets.all(16),
  child: Wrap(
    spacing: 10,
    runSpacing: 10,
    children: [
      OutlinedButton.icon(
        onPressed: widget.onOpenCamera,
        icon: const Icon(Icons.videocam),
        label: const Text('Camera'),
      ),
      OutlinedButton.icon(
        onPressed:
          widget.session.importFromGallery,
        icon: const Icon(Icons.video_library),
        label: const Text('Gallery'),
      ),
      FilledButton.icon(
        onPressed: project.clips.isEmpty ||
          widget.session.state ==
            CreatorStudioSessionState
              .rendering
          ? null
          : widget.session.render,
        icon: const Icon(Icons.movie_creation),
        label: const Text('Render'),
      ),
      FilledButton(
        onPressed: project.renderedPath == null
          ? null
          : widget.onContinueToUpload,
        child: const Text('Continue'),
      ),
    ],
  ),
),
),
),
PublicationGatePanel(
  result:
    widget.session.previewPublicationGate(),
),
),
),
),
);
}
}

```

Replace firstOrNull where unavailable:

```
VideoClipEdit? selected;
```

```

for (final clip in project.clips) {
  if (clip.id == _selectedClipId) {
    selected = clip;
    break;
  }
}

```

8. Upload Screen Controller

apps/mobile_flutter/lib/features/creator_studio/ controllers/creator_upload_screen_controller.dart

```
import 'dart:async';

import 'package:flutter/foundation.dart';

import '../data/upload_repository.dart';
import '../domain/creator_project.dart';
import '../services/creator_upload_coordinator.dart';

final class CreatorUploadScreenController
    extends ChangeNotifier {
  CreatorUploadScreenController({
    required CreatorUploadCoordinator coordinator,
    required UploadRepository uploads,
  }) : _coordinator = coordinator,
      _uploads = uploads;

  final CreatorUploadCoordinator _coordinator;
  final UploadRepository _uploads;

  String? _uploadId;
  VideoUploadState? _state;
  StreamSubscription<VideoUploadState>? _subscription;
  Object? _error;

  String? get uploadId => _uploadId;
  VideoUploadState? get state => _state;
  Object? get error => _error;

  Future<void> start(CreatorProject project) async {
    try {
      _uploadId = await _coordinator.begin(project);

      await _subscription?.cancel();

      _subscription = _uploads
        .watch(_uploadId!)
        .listen((state) {
          _state = state;
          notifyListeners();
        });
    } catch (error) {
      _error = error;
      notifyListeners();
      rethrow;
    }
  }

  Future<void> pause() async {
    final id = _uploadId;
    if (id != null) {
      await _uploads.pause(id);
    }
  }

  Future<void> resume() async {
    final id = _uploadId;
    if (id != null) {
      await _uploads.resume(id);
    }
  }

  Future<void> retry() async {
    final id = _uploadId;
    if (id != null) {
      await _uploads.retry(id);
    }
  }

  Future<void> cancel() async {
    final id = _uploadId;
    if (id != null) {
      await _uploads.cancel(id);
    }
  }

  @override
  void dispose() {
    _subscription?.cancel();
    super.dispose();
  }
}
```

9. Upload and Publication Screen

**apps/mobile_flutter/lib/features/creator_studio/
presentation/creator_upload_screen.dart**

```
import 'package:flutter/material.dart';

import '../controllers/creator_upload_screen_controller.dart';
import '../data/upload_repository.dart';
import '../domain/creator_project.dart';
import 'widgets/upload_progress_card.dart';

class CreatorUploadScreen extends StatefulWidget {
  const CreatorUploadScreen({
    required this.project,
    required this.controller,
    required this.onUploadComplete,
    super.key,
  });

  final CreatorProject project;
  final CreatorUploadScreenController controller;
  final VoidCallback onUploadComplete;

  @override
  State<CreatorUploadScreen> createState() =>
    _CreatorUploadScreenState();
}

class _CreatorUploadScreenState
  extends State<CreatorUploadScreen> {
  @override
  void initState() {
    super.initState();

    widget.controller
      ..addListener(_refresh)
      ..start(widget.project);
  }

  void _refresh() {
    if (!mounted) {
      return;
    }

    setState(() {});

    if (widget.controller.state?.status ==
        VideoUploadStatus.completed) {
      widget.onUploadComplete();
    }
  }

  @override
  void dispose() {
    widget.controller
      ..removeListener(_refresh)
      ..dispose();

    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    final state = widget.controller.state;

    return Scaffold(
      appBar: AppBar(
        title: const Text('Upload video'),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: state == null
          ? const Center(
              child: CircularProgressIndicator(),
            )
          : UploadProgressCard(
              state: state,
              onPause: widget.controller.pause,
              onResume: widget.controller.resume,
              onRetry: widget.controller.retry,
              onCancel: widget.controller.cancel,
            ),
      ),
    );
  }
}
```

```

    },
  );
}
}

```

10. Crashlytics Integration

apps/mobile_flutter/lib/features/creator_studio/services/creator_studio_observability.dart

```

import 'package:firebase_analytics/firebase_analytics.dart';
import 'package:firebase_crashlytics/firebase_crashlytics.dart';

```

```

final class CreatorStudioObservability {
  CreatorStudioObservability({
    FirebaseAnalytics? analytics,
    FirebaseCrashlytics? crashlytics,
  }) : _analytics =
        analytics ?? FirebaseAnalytics.instance,
        _crashlytics =
        crashlytics ?? FirebaseCrashlytics.instance;

```

```

  final FirebaseAnalytics _analytics;
  final FirebaseCrashlytics _crashlytics;

```

```

  Future<void> setContext({
    required String projectId,
    required String operation,
    required int clipCount,
  }) async {
    await _crashlytics.setCustomKey(
      'creator_project_id',
      projectId,
    );

    await _crashlytics.setCustomKey(
      'creator_operation',
      operation,
    );

    await _crashlytics.setCustomKey(
      'creator_clip_count',
      clipCount,
    );
  }

```

```

  Future<void> recordEvent(
    String name, {
    Map<String, Object>? parameters,
  }) {
    return _analytics.logEvent(
      name: name,
      parameters: parameters,
    );
  }

```

```

  Future<void> recordFailure(
    Object error,
    StackTrace stackTrace, {
    required String reason,
    required String projectId,
    required String operation,
  }) async {
    await setContext(
      projectId: projectId,
      operation: operation,
      clipCount: 0,
    );

    await _crashlytics.recordError(
      error,
      stackTrace,
      reason: reason,
      fatal: false,
    );
  }
}

```

Mandatory events:

```

creator_project_created
creator_camera_opened

```

```
creator_recording_started
creator_clip_recorded
creator_gallery_imported
creator_clip_trimmed
creator_clip_split
creator_clip_deleted
creator_clip_reordered
creator_undo
creator_redo
creator_draft_saved
creator_draft_recovered
creator_render_started
creator_render_completed
creator_render_failed
creator_upload_started
creator_upload_paused
creator_upload_resumed
creator_upload_cancelled
creator_upload_retried
creator_upload_completed
creator_publication_blocked
```

11. RC1 Route Gate

**apps/mobile_flutter/lib/features/creator_studio/
presentation/creator_studio_gate.dart**

```
import 'package:flutter/material.dart';

import '../../core/config/creator_studio_flags.dart';

class CreatorStudioGate extends StatelessWidget {
  const CreatorStudioGate({
    required this.child,
    super.key,
  });

  final Widget child;

  @override
  Widget build(BuildContext context) {
    if (!CreatorStudioFlags.enabled) {
      return const Scaffold(
        body: Center(
          child: Text(
            'Creator Studio is not enabled for this account.',
          ),
        ),
      );
    }

    return child;
  }
}
```

12. Mandatory Publication Enforcement

The mobile preview gate is not enough. Publishing must be blocked on the server.

backend/firebase_functions/src/creator-studio/rc1-publication-gate.ts

```
export interface Rc1PublicationEvidence {
  ownerId: string;
  projectId: string;
  uploadId: string;

  rendered: boolean;
  uploaded: boolean;
  processingComplete: boolean;
  moderationPassed: boolean;
  copyrightPassed: boolean;

  title: string;
  caption: string;
  captionsReviewed: boolean;

  advancedFeatures: {
    captions: boolean;
  }
}
```

```

    music: boolean;
    voiceOver: boolean;
    ecosystemObjects: boolean;
    aiEditExecution: boolean;
  };
}

export interface Rc1PublicationDecision {
  allowed: boolean;
  blockers: string[];
}

export function evaluateRc1PublicationGate(
  input: Rc1PublicationEvidence,
): Rc1PublicationDecision {
  const blockers: string[] = [];

  if (!input.rendered) {
    blockers.push("Local rendering is incomplete.");
  }

  if (!input.uploaded) {
    blockers.push("Video upload is incomplete.");
  }

  if (!input.processingComplete) {
    blockers.push("Server processing is incomplete.");
  }

  if (!input.moderationPassed) {
    blockers.push("Moderation has not passed.");
  }

  if (!input.copyrightPassed) {
    blockers.push("Copyright verification has not passed.");
  }

  if (!input.title.trim()) {
    blockers.push("A video title is required.");
  }

  if (!input.caption.trim()) {
    blockers.push("A publication caption is required.");
  }

  if (!input.captionsReviewed) {
    blockers.push("Publication metadata requires review.");
  }

  const uncertifiedFeaturesEnabled =
    input.advancedFeatures.captions ||
    input.advancedFeatures.music ||
    input.advancedFeatures.voiceOver ||
    input.advancedFeatures.ecosystemObjects ||
    input.advancedFeatures.aiEditExecution;

  if (uncertifiedFeaturesEnabled) {
    blockers.push(
      "One or more uncertified Creator Studio features are enabled.",
    );
  }

  return {
    allowed: blockers.length === 0,
    blockers,
  };
}

```

13. Backend RC1 Publish Function

backend/firebase_functions/src/creator-studio/publish-creator-video-rc1.ts

```

import {
  HttpsError,
  onCall,
} from "firebase-functions/v2/https";

import {
  FieldValue,
  getFirestore,
} from "firebase-admin/firestore";

import {

```

```

    evaluateRclPublicationGate,
  } from "./rcl-publication-gate";

const firestore = getFirestore();

interface PublishRclRequest {
  projectId: string;
  uploadId: string;
  title: string;
  caption: string;
  visibility: "public" | "followers" | "private";
  captionsReviewed: boolean;
}

export const publishCreatorVideoRcl = onCall(
  async (request) => {
    const uid = request.auth?.uid;

    if (!uid) {
      throw new HttpsError(
        "unauthenticated",
        "Authentication is required.",
      );
    }

    const input = request.data as PublishRclRequest;

    if (!input.projectId || !input.uploadId) {
      throw new HttpsError(
        "invalid-argument",
        "projectId and uploadId are required.",
      );
    }

    const uploadRef = firestore
      .collection("creatorUploadSessions")
      .doc(input.uploadId);

    const processingRef = firestore
      .collection("creatorProcessingJobs")
      .doc(input.uploadId);

    const [uploadSnapshot, processingSnapshot] =
      await Promise.all([
        uploadRef.get(),
        processingRef.get(),
      ]);

    if (!uploadSnapshot.exists) {
      throw new HttpsError(
        "not-found",
        "Upload session was not found.",
      );
    }

    const upload = uploadSnapshot.data()!;

    if (upload.ownerId !== uid) {
      throw new HttpsError(
        "permission-denied",
        "The upload belongs to another creator.",
      );
    }

    const processing =
      processingSnapshot.data() ?? {};

    const decision = evaluateRclPublicationGate({
      ownerId: uid,
      projectId: input.projectId,
      uploadId: input.uploadId,
      rendered: upload.rendered === true,
      uploaded: upload.status === "uploaded",
      processingComplete:
        processing.status === "completed",
      moderationPassed:
        processing.moderationStatus === "approved",
      copyrightPassed:
        processing.copyrightStatus === "approved",
      title: input.title ?? "",
      caption: input.caption ?? "",
      captionsReviewed:
        input.captionsReviewed === true,
      advancedFeatures: {
        captions: false,
        music: false,
        voiceOver: false,
      },
    });
  },
);

```

```

        ecosystemObjects: false,
        aiEditExecution: false,
      },
    ));

    if (!decision.allowed) {
      await firestore
        .collection("creatorPublicationAttempts")
        .add({
          ownerId: uid,
          projectId: input.projectId,
          uploadId: input.uploadId,
          allowed: false,
          blockers: decision.blockers,
          createdAt: FieldValue.serverTimestamp(),
        });

      throw new HttpsError(
        "failed-precondition",
        "Publishing requirements have not been met.",
        {
          blockers: decision.blockers,
        },
      );
    }
  }

  const videoRef =
    firestore.collection("videos").doc();

  await firestore.runTransaction(
    async (transaction) => {
      transaction.set(videoRef, {
        id: videoRef.id,
        ownerId: uid,
        userId: uid,
        projectId: input.projectId,
        uploadId: input.uploadId,
        title: input.title.trim(),
        caption: input.caption.trim(),
        visibility: input.visibility,
        status: "published",
        source: "creator-studio-1.1a-rc1",
        videoUrl: processing.videoUrl,
        hlsUrl: processing.hlsUrl,
        previewUrl: processing.previewUrl ?? null,
        thumbnailUrl:
          processing.thumbnailUrl ?? null,
        views: 0,
        likes: 0,
        shares: 0,
        comments: 0,
        createdAt: FieldValue.serverTimestamp(),
        updatedAt: FieldValue.serverTimestamp(),
      });

      transaction.update(uploadRef, {
        status: "published",
        videoId: videoRef.id,
        updatedAt: FieldValue.serverTimestamp(),
      });
    },
  );

  return {
    published: true,
    videoId: videoRef.id,
  };
},
);

```

14. Required Flutter Tests

test/features/creator_studio/creator_studio_flags_test.dart

```

import 'package:flutter_test/flutter_test.dart';

void main() {
  test(
    'RC1 advanced features remain disabled',
    () {
      expect(
        CreatorStudioFlags.captionsEnabled,
        isFalse,
      );
    },
  );
}

```

```

);

expect(
  CreatorStudioFlags.musicEnabled,
  isFalse,
);

expect(
  CreatorStudioFlags.voiceOverEnabled,
  isFalse,
);

expect(
  CreatorStudioFlags.ecosystemObjectsEnabled,
  isFalse,
);

expect(
  CreatorStudioFlags.aiEditExecutionEnabled,
  isFalse,
);
},
);
}

```

test/features/creator_studio/draft_recovery_test.dart

```

import 'package:flutter_test/flutter_test.dart';

void main() {
  test(
    'saved draft can be recovered with clip edits',
    () async {
      final repository =
        InMemoryCreatorProjectRepository();

      final project = CreatorProject(
        id: 'project-1',
        ownerId: 'creator-1',
        title: 'Pilot video',
        objective: CreatorObjective.entertain,
        status: CreatorProjectStatus.draft,
        clips: const [
          VideoClipEdit(
            id: 'clip-1',
            sourcePath: '/tmp/clip.mp4',
            originalDurationMs: 10000,
            trimStartMs: 1000,
            trimEndMs: 8000,
          ),
        ],
        audioTracks: const [],
        captions: const [],
        timelineObjects: const [],
        createdAt: DateTime(2026),
        updatedAt: DateTime(2026),
      );

      await repository.save(project);

      final recovered =
        await repository.findById('project-1');

      expect(recovered, isNotNull);
      expect(recovered!.clips, hasLength(1));
      expect(
        recovered.clips.first.trimStartMs,
        1000,
      );
      expect(
        recovered.clips.first.trimEndMs,
        8000,
      );
    },
  );
}

```

test/features/creator_studio/timeline_history_test.dart

```

import 'package:flutter_test/flutter_test.dart';

void main() {
  test(
    'undo and redo preserve split operation',
    () {

```

```

    final controller = CreatorEditorController(
      initialProject: testCreatorProject(),
    );

    controller.splitClip(
      clipId: 'clip-1',
      splitAtMs: 5000,
    );

    expect(controller.project.clips, hasLength(2));

    controller.undo();

    expect(controller.project.clips, hasLength(1));

    controller.redo();

    expect(controller.project.clips, hasLength(2));
  },
);

test(
  'delete can be undone',
  () {
    final controller = CreatorEditorController(
      initialProject: testCreatorProject(),
    );

    controller.deleteClip('clip-1');

    expect(controller.project.clips, isEmpty);

    controller.undo();

    expect(controller.project.clips, hasLength(1));
  },
);
}

```

test/features/creator_studio/publication_blocking_test.dart

```

import 'package:flutter_test/flutter_test.dart';

void main() {
  const gate = VideoPublicationGate();

  test(
    'blocks publishing before rendering',
    () {
      final result = gate.evaluate(
        const PublicationGateInput(
          hasRenderedVideo: false,
          uploadComplete: false,
          moderationPassed: false,
          copyrightPassed: false,
          metadataComplete: true,
          captionReviewComplete: true,
          validTimelineObjects: true,
          aiChecklistRequired: false,
          aiChecklistComplete: false,
        ),
      );

      expect(result.allowed, isFalse);

      expect(
        result.blockers,
        contains('Video rendering is incomplete.'),
      );
    },
  );

  test(
    'blocks publishing before moderation',
    () {
      final result = gate.evaluate(
        const PublicationGateInput(
          hasRenderedVideo: true,
          uploadComplete: true,
          moderationPassed: false,
          copyrightPassed: true,
          metadataComplete: true,
          captionReviewComplete: true,
          validTimelineObjects: true,
          aiChecklistRequired: false,
          aiChecklistComplete: false,
        ),
      );
    },
  );
}

```

```

    ),
  );
  expect(result.allowed, isFalse);
},
);
}

```

15. Backend Tests

backend/firebase_functions/test/creator-studio/rc1-publication-gate.test.ts

```

import {
  describe,
  expect,
  it,
} from "vitest";

import {
  evaluateRc1PublicationGate,
} from "../../src/creator-studio/rc1-publication-gate";

describe("Creator Studio RC1 publication gate", () => {
  it("blocks an unrendered project", () => {
    const result = evaluateRc1PublicationGate({
      ownerId: "creator-1",
      projectId: "project-1",
      uploadId: "upload-1",
      rendered: false,
      uploaded: true,
      processingComplete: true,
      moderationPassed: true,
      copyrightPassed: true,
      title: "Test video",
      caption: "Test caption",
      captionsReviewed: true,
      advancedFeatures: {
        captions: false,
        music: false,
        voiceOver: false,
        ecosystemObjects: false,
        aiEditExecution: false,
      },
    });

    expect(result.allowed).toBe(false);
    expect(result.blockers).toContain(
      "Local rendering is incomplete.",
    );
  });

  it("blocks uncertified advanced features", () => {
    const result = evaluateRc1PublicationGate({
      ownerId: "creator-1",
      projectId: "project-1",
      uploadId: "upload-1",
      rendered: true,
      uploaded: true,
      processingComplete: true,
      moderationPassed: true,
      copyrightPassed: true,
      title: "Test video",
      caption: "Test caption",
      captionsReviewed: true,
      advancedFeatures: {
        captions: true,
        music: false,
        voiceOver: false,
        ecosystemObjects: false,
        aiEditExecution: false,
      },
    });

    expect(result.allowed).toBe(false);
  });

  it("allows a complete RC1 publication", () => {
    const result = evaluateRc1PublicationGate({
      ownerId: "creator-1",
      projectId: "project-1",
      uploadId: "upload-1",

```

```

        rendered: true,
        uploaded: true,
        processingComplete: true,
        moderationPassed: true,
        copyrightPassed: true,
        title: "Test video",
        caption: "Test caption",
        captionsReviewed: true,
        advancedFeatures: {
            captions: false,
            music: false,
            voiceOver: false,
            ecosystemObjects: false,
            aiEditExecution: false,
        },
    },
});

expect(result.allowed).toBe(true);
expect(result.blockers).toHaveLength(0);
});
});
});

```

16. CI Validation Workflow

.github/workflows/creator-studio-rc1.yml

```

name: Creator Studio 1.1A RC1

on:
  pull_request:
    paths:
      - "apps/mobile_flutter/lib/features/creator_studio/**"
      - "apps/mobile_flutter/test/features/creator_studio/**"
      - "backend/firebase_functions/src/creator-studio/**"
      - "backend/firebase_functions/test/creator-studio/**"
      - ".github/workflows/creator-studio-rc1.yml"

  push:
    branches:
      - "feature/creator-studio-1.1a-rc1"
      - "release/creator-studio-1.1a-rc1"

jobs:
  flutter:
    runs-on: ubuntu-latest

    defaults:
      run:
        working-directory: apps/mobile_flutter

    steps:
      - uses: actions/checkout@v4

      - uses: subosito/flutter-action@v2
        with:
          channel: stable
          cache: true

      - name: Dependencies
        run: flutter pub get

      - name: Format
        run: >
          dart format
          --output=none
          --set-exit-if-changed
          lib
          test

      - name: Analyze
        run: flutter analyze

      - name: Tests
        run: flutter test

  functions:
    runs-on: ubuntu-latest

    defaults:
      run:
        working-directory: backend/firebase_functions

    steps:

```

- uses: actions/checkout@v4
 - uses: actions/setup-node@v4
with:
 - node-version: 20
 - cache: npm
 - cache-dependency-path:
 - backend/firebase_functions/package-lock.json
 - name: Dependencies
run: npm ci
 - name: Lint
run: npm run lint
 - name: Build
run: npm run build
 - name: Tests
run: npm test
-

17. Immutable Candidate Script

scripts/freeze_creator_studio_1_1a_rc1.sh

```
#!/usr/bin/env bash
set -euo pipefail

FEATURE_BRANCH="feature/creator-studio-1.1a-rc1"
RELEASE_BRANCH="release/creator-studio-1.1a-rc1"
TAG="yohpal-live-creator-studio-v1.1a-rc1"

CURRENT_BRANCH="$(git branch --show-current)"

if [[ "${CURRENT_BRANCH}" != "${FEATURE_BRANCH}" ]]; then
    echo "FAIL: expected ${FEATURE_BRANCH}, found ${CURRENT_BRANCH}"
    exit 1
fi

if [[ -n "$(git status --porcelain)" ]]; then
    echo "FAIL: working tree is not clean"
    git status --short
    exit 2
fi

bash scripts/validate_creator_studio_1_1a_rc1.sh

git checkout -b "${RELEASE_BRANCH}"

COMMIT="$(git rev-parse HEAD)"

git tag -a "${TAG}" \
    -m "YohPal Live Creator Studio 1.1A RC1 ${COMMIT}"

echo "Candidate frozen"
echo "Feature branch: ${FEATURE_BRANCH}"
echo "Release branch: ${RELEASE_BRANCH}"
echo "Commit: ${COMMIT}"
echo "Tag: ${TAG}"
```

18. Validation and Evidence Script

scripts/validate_creator_studio_1_1a_rc1.sh

```
#!/usr/bin/env bash
set -euo pipefail

EVIDENCE="release-evidence/creator-studio-1.1a-rc1"

mkdir -p \
    "${EVIDENCE}/01-repository" \
    "${EVIDENCE}/02-flutter" \
    "${EVIDENCE}/03-backend"

pushd apps/mobile_flutter >/dev/null

flutter clean

flutter pub get \
```

```

2>&1 |
tee "../../../${EVIDENCE}/02-flutter/pub-get.txt"

dart format \
  --output=none \
  --set-exit-if-changed \
  lib test \
2>&1 |
tee "../../../${EVIDENCE}/02-flutter/format.txt"

flutter analyze \
2>&1 |
tee "../../../${EVIDENCE}/02-flutter/analyze.txt"

flutter test \
2>&1 |
tee "../../../${EVIDENCE}/02-flutter/test.txt"

popd >/dev/null

pushd backend/firebase_functions >/dev/null

npm ci \
2>&1 |
tee "../../../${EVIDENCE}/03-backend/npm-ci.txt"

npm run lint \
2>&1 |
tee "../../../${EVIDENCE}/03-backend/lint.txt"

npm run build \
2>&1 |
tee "../../../${EVIDENCE}/03-backend/build.txt"

npm test \
2>&1 |
tee "../../../${EVIDENCE}/03-backend/test.txt"

popd >/dev/null

git branch --show-current \
  > "${EVIDENCE}/01-repository/branch.txt"

git rev-parse HEAD \
  > "${EVIDENCE}/01-repository/commit.txt"

git status --short \
  > "${EVIDENCE}/01-repository/status.txt"

git tag --points-at HEAD \
  > "${EVIDENCE}/01-repository/tags.txt"

echo "PASS: source validation complete"

```

19. Evidence Directory

```

release-evidence/
├── creator-studio-1.1a-rc1/
│   ├── manifest.json
│   ├── 01-repository/
│   ├── 02-flutter/
│   ├── 03-backend/
│   ├── 04-android-camera/
│   ├── 05-ios-camera/
│   ├── 06-multi-clip/
│   ├── 07-timeline/
│   ├── 08-draft-recovery/
│   ├── 09-rendering/
│   ├── 10-upload/
│   ├── 11-publication-gate/
│   ├── 12-crashlytics/
│   ├── 13-defects/
│   └── 14-governance/

```

release-evidence/creator-studio-1.1a-rc1/manifest.json

```

{
  "release": "YOHPAL LIVE CREATOR STUDIO 1.1A-RC1",
  "candidate": {
    "featureBranch": "feature/creator-studio-1.1a-rc1",

```

```

    "releaseBranch": "release/creator-studio-1.1a-rc1",
    "commitHash": "REPLACE_WITH_FULL_40_CHARACTER_SHA",
    "tag": "yohpal-live-creator-studio-v1.1a-rc1",
    "androidBuild": "REPLACE",
    "iosBuild": "REPLACE"
  },
  "advancedFeatures": {
    "captions": false,
    "music": false,
    "voiceOver": false,
    "ecosystemObjects": false,
    "aiEditExecution": false
  },
  "gates": [
    {
      "id": "REPOSITORY",
      "mandatory": true,
      "status": "PENDING",
      "evidence": ["01-repository/"]
    },
    {
      "id": "FLUTTER",
      "mandatory": true,
      "status": "PENDING",
      "evidence": ["02-flutter/"]
    },
    {
      "id": "BACKEND",
      "mandatory": true,
      "status": "PENDING",
      "evidence": ["03-backend/"]
    },
    {
      "id": "ANDROID_CAMERA",
      "mandatory": true,
      "status": "PENDING",
      "evidence": ["04-android-camera/"]
    },
    {
      "id": "IOS_CAMERA",
      "mandatory": true,
      "status": "PENDING",
      "evidence": ["05-ios-camera/"]
    },
    {
      "id": "MULTI_CLIP",
      "mandatory": true,
      "status": "PENDING",
      "evidence": ["06-multi-clip/"]
    },
    {
      "id": "TIMELINE",
      "mandatory": true,
      "status": "PENDING",
      "evidence": ["07-timeline/"]
    },
    {
      "id": "DRAFT_RECOVERY",
      "mandatory": true,
      "status": "PENDING",
      "evidence": ["08-draft-recovery/"]
    },
    {
      "id": "RENDERING",
      "mandatory": true,
      "status": "PENDING",
      "evidence": ["09-rendering/"]
    },
    {
      "id": "UPLOAD",
      "mandatory": true,
      "status": "PENDING",
      "evidence": ["10-upload/"]
    },
    {
      "id": "PUBLICATION_GATE",
      "mandatory": true,
      "status": "PENDING",
      "evidence": ["11-publication-gate/"]
    },
    {
      "id": "CRASHLYTICS",
      "mandatory": true,
      "status": "PENDING",
      "evidence": ["12-crashlytics/"]
    },
    {

```

```

    "id": "DEFECTS",
    "mandatory": true,
    "status": "PENDING",
    "evidence": [ "13-defects/" ]
  }
}
```

20. Physical Device Validation Matrix

Test	Android	iPhone
Camera initialization	Required	Required
Front/rear switching	Required	Required
Permission denial/recovery	Required	Required
Record first clip	Required	Required
Record multiple clips	Required	Required
Pause/resume	Required	Required
Delete last clip	Required	Required
Gallery import	Required	Required
Trim	Required	Required
Split	Required	Required
Delete	Required	Required
Reorder	Required	Required
Undo/redo	Required	Required
Draft recovery	Required	Required
Local rendering	Required	Required
Upload progress	Required	Required

Cancel upload	Required	Required
Retry failed upload	Required	Required
Publication blocking	Required	Required
Crashlytics	Required	Required

21. Release Decision Rules

GO

Requires:

- Android and iPhone camera proof
- Multi-clip recording passes
- Gallery import passes
- Timeline operations pass
- Draft recovery after force-close passes
- Rendered output has synchronized audio and video
- Upload cancel and retry pass
- Server publication gate blocks incomplete media
- Advanced features remain disabled
- No unresolved Critical or P0 defect
- Flutter and backend pipelines pass
- Crashlytics is visible

CONDITIONAL GO

Allowed only when:

- All P0 functionality passes
- No crash, data-loss or publication-security defect remains
- Remaining issue is non-critical
- Pilot remains allow-listed
- A named owner and correction date are recorded

NO-GO

Mandatory when:

- Camera fails on either platform
- Recorded clips are lost
- Draft recovery fails
- Rendering produces corrupt, silent or desynchronized output
- Upload cannot be cancelled or retried
- Unprocessed content can be published
- Uncertified advanced features become accessible
- Crashlytics is unavailable
- Evidence does not match the immutable candidate

Current Release Position

Implementation: authorized.

Pilot activation: permitted only after source validation and internal QA build generation.

Public rollout: not authorized until physical Android and iPhone evidence passes.

Advanced features: remain disabled.

Next Smart Move

EXECUTE: YOHPAL LIVE CREATOR STUDIO RELEASE 1.1A-RC1 PHYSICAL DEVICE VALIDATION

Use only one immutable RC1 commit and tag.

QA must return:

1. Android camera and multi-clip recording proof.
2. iPhone camera and multi-clip recording proof.
3. Gallery import proof.
4. Trim, split, delete and reorder proof.
5. Undo and redo proof.
6. Draft autosave and force-close recovery proof.
7. Rendered output with audio/video synchronization proof.
8. Upload progress, cancellation and retry proof.
9. Server-side publication blocking proof.
10. Proof that captions, music, voice-over, ecosystem objects and AI edit execution remain disabled.
11. Crashlytics evidence.
12. Flutter analysis and test outputs.
13. Backend build and test outputs.
14. Defect register.
15. Final GO, CONDITIONAL GO or NO-GO recommendation.

Do not activate the advanced Creator Studio phases until RC1 passes the complete camera-to-render-to-upload validation journey on both Android and iPhone.